

3. Buffers

Los buffers son herramientas ampliamente empleadas en la electrónica digital o en la computación. Un buffer es un dispositivo o región de memoria que permite el almacenamiento temporal de datos dentro de un sistema digital.

La mayoría de las funciones de los buffers son transparentes para los usuarios de sistemas digitales, pero permiten optimizar rendimiento su ofrecen una mejor experiencia de usuario.

Vamos a ilustrar el uso de un buffer con algo que es familiar a todo el mundo, el consumo de contenido multimedia, por ejemplo, viendo un vídeo de Youtube, este debe llegar a tu equipo desde los servidores y mostrarse en tu equipo a una velocidad fija.

En este escenario debemos obtener datos de una fuente y mostrar dichos datos en el destino (herramienta de reproducción del vídeo para que se emita por la pantalla y los altavoces). En este escenario imaginemos que el vídeo ocupa 100MB y que la calidad del vídeo es de 5MB por segundo, siendo un vídeo de 20 segundos de duración en el que la información de cada segundo son 5MB. Si quiero reproducir el vídeo a su velocidad natural, necesitare que me llegue esta información con una cadencia exacta de 5MB por segundo, para que mi reproductor tenga esa información en el momento exacto. Pero el vídeo no se reproduce segundo a segundo, si no frame a frame, o muestra de audio a muestra de audio. En este escenario, se requeriría una sincronización óptima y una latencia nula para poder recibir la información de los servidores y mostrarla.

Este escenario no es realista, por ello se usa un buffer para gestionar esa información. Nuestro dispositivo solicitará la información del vídeo al servidor, y este se lo mandará por bloques, este envío tendrá incertidumbres basadas en la cantidad de peticiones a los servidores, el ancho de banda de la conexión, la congestión en diferentes tramos de las redes por las que pase la información, etc. Si mi dispositivo tiene un buffer en el que puede almacenar la información que le llegue, y puede ir consumiendo dicha información para mostrar el vídeo, existirán dos procesos de entrada y salida de información que no tendrán una gran sensibilidad, el buffer se irá llenando en función de las capacidades de servicio de la información, y el dispositivo irá consumiendo dicha información a la cadencia necesaria para mostrar el vídeo de forma homogénea y satisfactoria para el usuario.

Este tipo de comportamiento se replica en muchos escenarios, sobre todo en aquellos donde el tiempo de las operaciones no sea determinista. Entender este concepto es importante para entender la utilidad de los buffers. El estudio en profundidad de este mecanismo está fuera del alcance del módulo, pero tener una concepción del problema que resuelven es importante para entender su uso.

En Java existen clases abstractas e interfaces que permiten realizar estructuras de datos para emular este comportamiento, pero están fuera del alcance de este curso. Nos centraremos en trabajar con dos clases específicas que permiten leer y escribir en ficheros de forma optimizada gracias a buffers.

BufferedWriter y BufferedReader

Estas dos clases (BufferedWriter y BufferedReader) pertenecen al paquete `java.io` y proporcionan un buffer de almacenamiento para escritura y lectura, con el objetivo de mejorar el rendimiento de las operaciones de entrada y salida, ya que minimizan los accesos a recursos del sistema, al leer bloques de información más grandes que luego se consumen al ritmo necesitado por las aplicaciones.

Ejercicios

Ejercicio 1.

Enlace a El Quijote de Cervantes en Gutenberg:

<https://www.gutenberg.org/cache/epub/2000/pg2000.txt>

Realiza una lista de las 100 palabras más repetidas en el libro. Para contar las palabras usa la colección o colecciones óptimas y optimiza la lectura del fichero.

Nota: Puedes suponer que las palabras junto a signos de puntuación o acentos son diferentes, por ejemplo, que y ¿que pueden ser palabras diferentes, como y cómo pueden ser diferentes.